
TP 2.1 - GENERADORES PSEUDOALEATORIOS

Constante Gonzalo

Universidad Tecnologica Nacional
Facultad Regional Rosario
Zeballos 1341, S2000, Argentina
constante.gonzalo2@gmail.com

Gil Francisco Marcelo

Universidad Tecnologica Nacional
Facultad Regional Rosario
Zeballos 1341, S2000, Argentina
fran17mg@gmail.com

Ferrero Santiago

Universidad Tecnologica Nacional
Facultad Regional Rosario
Zeballos 1341, S2000, Argentina
santifnob@gmail.com

7 de junio de 2026

ABSTRACT

El presente documento detalla el desarrollo e implementación de algoritmos deterministas para la generación de secuencias de números pseudoaleatorios bajo el lenguaje de programación Python 3.x, en el marco de la cátedra de Simulación. El objetivo central radica en la construcción de un Generador Congruencial Lineal (GCL) y su posterior contrastación frente a otros métodos probabilísticos, incluyendo el generador nativo del lenguaje. Para validar la calidad estadística y la hipótesis de uniformidad e independencia de las secuencias obtenidas, se exponen los resultados de cuatro pruebas estadísticas fundamentales: el test de Chi-cuadrado, el test de rachas, el test de series y el test del póker. Finalmente, se analizan las implicancias de la selección de parámetros en el período del generador mediante herramientas de visualización bidimensional y tablas comparativas.

Keywords Simulación · Generadores pseudoaleatorios · GCL · Pruebas estadísticas

1. Introducción

En el campo de la simulación por computadora y los métodos de Montecarlo, la disponibilidad de fuentes de aleatoriedad confiables y eficientes representa un pilar fundamental. Un número puramente aleatorio es aquel cuyo valor no puede ser predicho y proviene generalmente de procesos físicos de naturaleza estocástica (True Random Number Generators o TRNG). Sin embargo, debido a la necesidad de reproducibilidad en la experimentación científica y a las limitaciones de velocidad del hardware, la disciplina recurre mayoritariamente a los denominados números pseudoaleatorios.

Un número pseudoaleatorio se define como el resultado de un proceso algorítmico determinista que, partiendo de unas condiciones iniciales conocidas como semilla, genera secuencias numéricas que exhiben propiedades estadísticas idénticas a las del azar, careciendo de patrones o regularidades aparentes. La naturaleza determinista del algoritmo garantiza que, ante la replicación de la semilla original, la secuencia producida será exactamente la misma en cada ejecución.

Uno de los algoritmos más extendidos e históricamente relevantes es el Generador Congruencial Lineal (GCL), propuesto por Derrick Lehmer. Este método se basa en una relación de recurrencia modular de la forma:

$$X_{n+1} = (aX_n + c) \pmod{m} \quad (1)$$

Donde X_0 representa la semilla inicial, a es el multiplicador, c es la constante aditiva (o incremento) y m denota el módulo del generador. Los valores obtenidos se normalizan en el intervalo $[0, 1)$ mediante la relación $U_n = X_n/m$.

La calidad de un GCL está intrínsecamente ligada a la elección de estas constantes, las cuales determinan el período máximo antes de que la secuencia comience a repetirse cíclicamente. Si la selección de parámetros es deficiente, el generador manifestará correlaciones internas o una distribución no uniforme, anomalías capaces de invalidar por completo los resultados de cualquier modelo de simulación dependiente de ellos.

Por consiguiente, este trabajo aborda no solo la codificación de dichos generadores, sino también la ejecución de rigurosos tests estadísticos diseñados para evaluar empíricamente si las propiedades de uniformidad e independencia se sostienen con el nivel de significación requerido.

2. Marco teórico

2.1. Generación Estocástica vs. Determinista

En el modelado y simulación computacional de eventos discretos y métodos de Monte Carlo, la disponibilidad de fuentes confiables de variabilidad estocástica constituye un requisito crítico. Las metodologías para proveer esta variabilidad se dividen conceptualmente en dos paradigmas mutuamente excluyentes: la generación estocástica (física) y la generación determinista (algorítmica).

2.1.1. Generadores de Números Aleatorios Reales (TRNG)

Los Generadores de Números Aleatorios Reales (*True Random Number Generators*, TRNG) cosechan entropía directamente a partir de procesos físicos intrínsecamente impredecibles y no deterministas. A nivel de hardware, se aprovechan fenómenos de la mecánica cuántica y la termodinámica, tales como:

- El decaimiento radiactivo de isótopos inestables detectado mediante contadores Geiger.
- El ruido térmico (ruido de Johnson-Nyquist) originado por el movimiento browniano de los portadores de carga eléctrica en resistencias y semiconductores.
- El ruido de disparo (*shot noise*) en uniones de semiconductores de silicio, el cual sirvió de base histórica para la máquina de cifrado y lotería gubernamental *ERNIE* (Electronic Random Number Indicator Equipment).
- El ruido atmosférico capturado por receptores de radio analógicos libres de filtros de sintonización, principio fundacional de la plataforma científica de acceso público *RANDOM.ORG*.

Matemáticamente, la salida de un TRNG representa una variable aleatoria real cuya realización es teóricamente imposible de modelar de forma analítica exacta a priori, asegurando que la entropía de Shannon del flujo sea máxima. No obstante, en el ámbito de la ingeniería de simulación, los TRNG presentan severos inconvenientes operativos:

1. **Inviabilidad de reproducción:** No es posible reproducir exactamente la misma secuencia de eventos estocásticos para realizar análisis de sensibilidad, depuración de algoritmos o comparación directa de políticas alternativas de diseño en igualdad de condiciones temporales (técnica de variables antitéticas o números aleatorios comunes).
2. **Limitaciones físicas de velocidad:** La tasa de generación de bits aleatorios se encuentra acotada por las constantes físicas de respuesta del hardware transductor, resultando ineficiente para simulaciones de gran escala que demandan miles de millones de muestras por segundo.
3. **Riesgo de sesgo por deriva:** Los componentes físicos del hardware envejecen, sufren variaciones por temperatura o interferencias electromagnéticas, lo que puede inducir sutiles sesgos estadísticos acumulativos en las muestras obtenidas.

2.1.2. Generadores de Números Pseudoaleatorios (PRNG)

Frente a las limitaciones de los TRNG, la computación moderna emplea Generadores de Números Pseudoaleatorios (*Pseudorandom Number Generators*, PRNG). Un PRNG es un algoritmo estrictamente determinista estructurado a partir de una relación de recurrencia recursiva de la forma:

$$S_{n+1} = f(S_n) \quad (2)$$

Donde S_n representa el vector de estado interno del generador en el paso discreto n , y f es una función de transición de estados predefinida. La inicialización del algoritmo se realiza mediante un valor o conjunto de valores denominado *Semilla* (X_0 o S_0).

A partir de una semilla idéntica, el generador reproducirá exactamente la misma secuencia matemática de números. Si la secuencia producida $\{U_1, U_2, \dots\}$ imita con precisión el comportamiento de una sucesión de variables aleatorias independientes e idénticamente distribuidas (*i.i.d.*) bajo una ley uniforme continua, se considera un PRNG de alta fidelidad para simulación.

2.2. Algoritmos de Generación Pseudoaleatoria

2.2.1. Método de los Cuadrados Medios (Middle-Square)

Concebido históricamente por John von Neumann y Nicholas Metropolis en 1946 durante sus investigaciones aplicadas al transporte de neutrones en el Proyecto Manhattan, constituye el primer algoritmo computacional diseñado para simular variables estocásticas.

El algoritmo opera sobre una aritmética decimal de longitud de dígitos fija, denotada por k , la cual debe ser un número par. El procedimiento recursivo consta de las siguientes etapas:

1. Se inicializa con una semilla entera X_0 compuesta exactamente de k dígitos.
2. Se calcula el cuadrado del valor actual de la secuencia para dar lugar a un valor intermedio de hasta $2k$ dígitos:

$$Y_n = X_n^2 \quad (3)$$

3. Si la longitud decimal de Y_n es estrictamente menor a $2k$, se efectúa un rellenado posicional con ceros a la izquierda hasta completar exactamente la dimensión $2k$.
4. Se extraen los k dígitos ubicados en la posición central de Y_n . Este bloque constituye el siguiente número de la secuencia, X_{n+1} .

Matemáticamente, la función de transición que aísla los dígitos centrales en base 10 se modela mediante la siguiente expresión analítica:

$$X_{n+1} = \left\lfloor \frac{X_n^2}{10^{k/2}} \right\rfloor \pmod{10^k} \quad (4)$$

A pesar de su relevancia histórica, este método presenta fallos dinámicos e inherentes muy severos:

- **Estados absorbentes (puntos fijos nulos):** Si la extracción de dígitos centrales produce un valor que contenga ceros secuenciales en su zona interna, el generador colapsará instantáneamente hacia el cero absoluto de forma definitiva ($X_{n+m} = 0 \forall m \geq 0$), ya que $0^2 = 0$.
- **Degeneración en ciclos cortos:** No existe una teoría matemática formal que permita predecir la longitud de los ciclos o períodos a partir de las propiedades aritméticas de la semilla X_0 . La inmensa mayoría de las elecciones de semilla conducen rápidamente a órbitas cerradas de período excesivamente corto.

2.2.2. Generador Congruencial Lineal (GCL)

Propuesto por Derrick H. Lehmer en 1949, el Generador Congruencial Lineal representa la estructura algorítmica más estudiada y difundida para la producción de enteros pseudoaleatorios de bajo coste computacional. Su funcionamiento se rige por la siguiente relación de recurrencia modular de primer orden:

$$X_{n+1} = (a \cdot X_n + c) \pmod{m} \quad (5)$$

Donde los parámetros del diseño algebraico deben satisfacer rigurosamente las siguientes definiciones de dominio:

- m : El **módulo**, un entero positivo ($m > 0$) que establece la escala del espacio discreto de estados posibles.
- a : El **multiplicador**, un entero restringido al intervalo $0 < a < m$.
- c : El **incremento**, un entero que satisface $0 \leq c < m$. Si $c > 0$, el algoritmo se define como un *generador congruencial mixto*. Si $c = 0$, el algoritmo se reduce a un *generador congruencial multiplicativo*.
- X_0 : La **semilla**, un entero inicial tal que $0 \leq X_0 < m$. En el caso multiplicativo ($c = 0$), se requiere la condición de coprimidad $\gcd(X_0, m) = 1$ para evitar que la secuencia colapse permanentemente en el estado nulo.

Proceso de Normalización: La secuencia de enteros $\{X_n\}$ producida por el GCL se sitúa en el dominio discreto $[0, m - 1]$. Para aproximar la variable uniforme continua estándar requerida por la mayoría de las transformaciones estocásticas ($U_n \sim U[0, 1)$), se efectúa una división aritmética por la magnitud del módulo divisor:

$$U_n = \frac{X_n}{m} \quad \text{donde} \quad U_n \in [0, 1) \quad (6)$$

Teorema de Hull-Dobell para el Período Máximo: Debido al principio del palomar en aritmética modular, cualquier secuencia determinista definida en un conjunto finito es inherentemente periódica. El período máximo posible para un GCL es exactamente m . El **Teorema de Hull-Dobell (1962)** establece de forma analítica que un generador congruencial lineal de tipo mixto ($c > 0$) alcanza un período completo o máximo de longitud m si y solo si se satisfacen simultáneamente tres condiciones algebraicas independientes:

1. **Coprimidad de paso:** El incremento c y el módulo m deben ser primos relativos:

$$\gcd(c, m) = 1 \quad (7)$$

2. **Divisibilidad por factores primos:** Para cada número primo p que sea divisor del módulo m ($p \mid m$), el multiplicador a debe satisfacer la relación de congruencia:

$$a \equiv 1 \pmod{p} \quad (8)$$

3. **Condición de potencia binaria:** Si el módulo m es divisible por 4 ($m \equiv 0 \pmod{4}$), el término multiplicativo debe cumplir estrictamente que:

$$a \equiv 1 \pmod{4} \quad (9)$$

La justificación matemática de la tercera condición radica en la estructura algebraica del grupo multiplicativo de residuos modulares impar $\mathbb{Z}_{2^k}^*$ (para potencias de dos habituales en arquitecturas de hardware). Dicho grupo no es cíclico para potencias de dos de exponente superior o igual a 3, sino que presenta una descomposición isomorfa al producto directo de dos grupos cíclicos independientes:

$$\mathbb{Z}_{2^k}^* \cong C_2 \times C_{2^{k-2}} \quad (10)$$

Al imponer la restricción $a \equiv 1 \pmod{4}$ en lugar de $a \equiv 3 \pmod{4}$, se inhibe la subdivisión de la trayectoria del generador en sub-ciclos aislados dentro de la estructura modular, forzando al operador lineal a alternar uniformemente las transiciones de bit entre estados de paridad par e impar.

En generadores multiplicativos ($c = 0$), al no cumplirse la condición $\gcd(c, m) = 1$, el período máximo obtenible es menor a m :

- Para un módulo binario de la forma $m = 2^k$ (con $k \geq 3$), el período máximo admisible es de apenas $m/4$, restricción que requiere que el multiplicador cumpla la congruencia $a \equiv 3 \pmod{5}$ o $a \equiv 5 \pmod{8}$, y que la semilla inicial X_0 sea impar.
- Para un módulo que sea un número primo m , el período máximo posible es $m - 1$, el cual se alcanza si y solo si el multiplicador a se selecciona como una raíz primitiva del primo m , forzando a la órbita del generador a barrer todos los residuos a excepción del cero absoluto (el cual actúa como un punto fijo absorbente).

Parámetros de GCL en Entornos de Producción Real: La Tabla 1 describe las elecciones estándar de constantes algebraicas empleadas históricamente por librerías informáticas clásicas para sus respectivas implementaciones del GCL.

Cuadro 1: Parámetros estructurales estándar del Generador Congruencial Lineal.

Entorno de Sistema	Módulo (m)	Multiplicador (a)	Incremento (c)	Operación de Salida
glibc (GCC / Linux)	2^{31}	1103515245	12345	Retorna bits [30 – 16]
Microsoft Visual C/C++	2^{31}	214013	2531011	Retorna bits [30 – 16]
Java (java.util.Random)	2^{48}	25214903917	11	Actualización con máscara de 48 bits
MATLAB (Legacy)	$2^{31} - 1$	16807	0	Generador Multiplicativo Puro
RANDU (IBM System/360)	2^{31}	65539	0	Defectuoso en 3D (Marsaglia)

2.2.3. Mersenne Twister (MT19937)

Formulado por Makoto Matsumoto y Takuji Nishimura en 1997, el algoritmo *Mersenne Twister* representa el motor por defecto en entornos de análisis de datos científicos modernos, tales como el módulo `random` nativo de Python 3.x.

Matemáticamente, el algoritmo opera como un Registro de Desplazamiento con Retroalimentación Generalizada Lineal (*Twisted Generalized Feedback Shift Register*, TGFSR) estructurado sobre el campo de Galois finito de dos elementos $\mathbb{F}_2$. El algoritmo MT19937 se rige por los siguientes parámetros estructurales:

- $w = 32$: El tamaño en bits de la palabra de cómputo del estado interno.
- $n = 624$: El tamaño del vector de estado del registro, lo que define una dimensión de memoria interna de 624×32 bits.
- $m = 397$: La distancia de desfase de la recurrencia o término medio de acoplamiento matricial.
- $r = 31$: El punto de enmascaramiento binario para la separación de bits.

Propiedades y Ventajas estadísticas del MT19937:

1. **Período Gigantesco:** La longitud de su período cíclico es un número primo de Mersenne equivalente a:

$$P = 2^{19937} - 1 \quad (11)$$

Esta colosal magnitud matemática garantiza que el ciclo de repetición del generador jamás se agote bajo ningún diseño de simulación computacional actual o futura.

2. **Equidistribución de alta dimensión:** El algoritmo garantiza la equidistribución teórica de sus vectores de números pseudoaleatorios hasta en 623 dimensiones consecutivas. Esto previene de forma absoluta la aparición del efecto de red tridimensional o bidimensional que invalida a los generadores lineales más sencillos.

Limitaciones Teóricas del MT19937:

- **Predecibilidad matemática:** No es un generador criptográficamente seguro (CSPRNG). Puesto que su ecuación de evolución temporal se define mediante transformaciones matriciales lineales sobre $\mathbb{F}_2$, un observador externo que obtenga de forma sucesiva exactamente 624 palabras de salida de 32 bits puede reconstruir algebraicamente la matriz de transición de estados y predecir todos los valores futuros de la secuencia de forma determinista.
- **Inercia ante propagación de ceros:** Si se inicializa con una semilla con una densidad de bits “0” excesivamente elevada, el generador requiere de varios miles de transiciones para superar temporalmente un sesgo de densidad inicial en su salida.

2.3. Defectos Geométricos Estructurales del GCL: El Teorema de Marsaglia

En 1968, George Marsaglia publicó un célebre artículo titulado “*Random numbers fall mainly in the planes*”, demostrando matemáticamente que los números pseudoaleatorios generados por cualquier GCL, lejos de dispersarse de forma homogénea en el espacio, se encuentran confinados sobre una familia reducida de hiperplanos paralelos dentro de un hipercubo unitario de dimensión arbitraria d .

2.3.1. Enunciado Matemático del Teorema de Marsaglia

Si se construyen tuplas consecutivas de dimensión d empleando valores normalizados generados por un GCL, de la forma:

$$\pi_i = (U_i, U_{i+1}, \dots, U_{i+d-1}) \in [0, 1)^d \quad (12)$$

El teorema establece que todos estos puntos vectoriales se sitúan obligatoriamente sobre una cantidad finita de hiperplanos paralelos del espacio d -dimensional. La cantidad máxima de hiperplanos paralelos, denotada por H , se encuentra acotada rígidamente por la desigualdad:

$$H \leq (d! \cdot m)^{1/d} \quad (13)$$

2.3.2. Demostración Formal del Teorema en Cuatro Pasos

Paso 1: Establecimiento de la Congruencia Lineal Modular. Dada la relación de recurrencia multiplicativa básica $X_{i+1} \equiv a \cdot X_i \pmod{m}$ (con $c = 0$), se busca demostrar que existe un vector de coeficientes enteros $\vec{c} = (c_1, c_2, \dots, c_d)$ no todos nulos de forma tal que:

$$c_1 + c_2 \cdot a + c_3 \cdot a^2 + \dots + c_d \cdot a^{d-1} \equiv 0 \pmod{m} \quad (14)$$

Multiplicando la congruencia anterior por el término de estado X_i y aplicando de forma recursiva la propiedad del generador, se deduce directamente la relación de combinación lineal entera sobre la secuencia:

$$c_1 \cdot X_i + c_2 \cdot X_{i+1} + \dots + c_d \cdot X_{i+d-1} \equiv 0 \pmod{m} \quad (15)$$

Esta congruencia modular implica la existencia de un entero I para el cual se satisface la igualdad:

$$c_1 \cdot X_i + c_2 \cdot X_{i+1} + \dots + c_d \cdot X_{i+d-1} = I \cdot m \quad (16)$$

Dividiendo ambos miembros de la ecuación por la magnitud del módulo divisor m para forzar la normalización de la secuencia ($U_n = X_n/m$), se obtiene la relación lineal fundamental:

$$c_1 \cdot U_i + c_2 \cdot U_{i+1} + \dots + c_d \cdot U_{i+d-1} = I \quad (17)$$

Donde $I \in \mathbb{Z}$ para todo índice temporal de la secuencia.

Paso 2: Definición Geométrica de la Familia de Hiperplanos. La Ecuación 17 define formalmente una familia de hiperplanos de dimensión d ortogonales al vector de coeficientes $\vec{c}$. Por ende, todos los puntos de la tupla π_i están obligados a ubicarse sobre el plano correspondiente a un valor entero del parámetro I .

Paso 3: Acotación del Número de Planos Intersecantes. La cantidad máxima de hiperplanos ortogonales a $\vec{c}$ que intersecan la región cerrada de frontera del hipercubo unitario $[0, 1)^d$ se encuentra acotada por la suma de los valores absolutos de las coordenadas de su vector normal:

$$\text{Número de planos} \leq \sum_{j=1}^d |c_j| \quad (18)$$

Paso 4: Aplicación de la Geometría de Números de Minkowski. De acuerdo con el teorema fundamental de Hermann Minkowski sobre las formas lineales convexas en la geometría de números, se garantiza que para cualquier valor real positivo de volumen espacial, siempre es posible encontrar un punto reticular de coordenadas enteras no nulas tales que la suma de sus formas lineales sea menor que la raíz d -ésima del volumen del módulo. En consecuencia, siempre existe una combinación de coeficientes enteros c_j que satisfacen el Paso 1 y verifican de forma estricta la cota superior:

$$\sum_{j=1}^d |c_j| \leq (d! \cdot m)^{1/d} \quad (19)$$

Esta demostración revela que, a medida que aumenta la dimensión del espacio d , la separación de los hiperplanos paralelos se incrementa exponencialmente, dejando inmensas áreas del hipercubo unitario completamente vacías (sin muestreo), lo que deforma el comportamiento estocástico de las simulaciones multidimensionales.

2.3.3. El Caso de Estudio de RANDU

El ejemplo histórico más destructivo de las deficiencias geométricas explicadas por el Teorema de Marsaglia lo representa el generador *RANDU*. Introducido de forma masiva en la arquitectura IBM System/360, se configuraba mediante los parámetros $m = 2^{31}$, $c = 0$, $a = 65539$.

El multiplicador de RANDU se diseñó con el fin de agilizar los cálculos mediante registros de desplazamiento binarios de ensamblador, dado que:

$$a = 2^{16} + 3 \quad (20)$$

Si se analiza algebraicamente el comportamiento de este término elevado al cuadrado en base al módulo:

$$(a - 3) = 2^{16} \implies (a - 3)^2 = 2^{32} \equiv 0 \pmod{2^{31}} \quad (21)$$

Desarrollando la expresión binomial:

$$a^2 - 6a + 9 \equiv 0 \pmod{2^{31}} \quad (22)$$

Multiplicando la congruencia anterior por un elemento del estado X_i y dividiendo por el módulo 2^{31} para normalizar, se obtiene que la totalidad de las ternas ordenadas de números pseudoaleatorios generadas por RANDU en dimensión $d = 3$ satisfacen rigurosamente la ecuación:

$$9U_i - 6U_{i+1} + U_{i+2} = I \quad \text{donde } I \in \mathbb{Z} \quad (23)$$

En este caso, el vector de coeficientes normales es $\vec{c} = (9, -6, 1)$, cuya suma de magnitudes absolutas es $|9| + |-6| + |1| = 16$. Esto implica analíticamente que la totalidad de los miles de millones de puntos generados por RANDU caen confinadas en, a lo sumo, apenas 15 planos paralelos dentro del cubo tridimensional $[0, 1)^3$.

2.4. Pruebas Estadísticas de Calidad (4 Tests Fundamentales)

Para certificar que un algoritmo PRNG genere secuencias numéricas aptas para estudios de simulación, estas deben ser sometidas a una validación empírica formal de sus propiedades estocásticas de uniformidad e independencia espacial y secuencial.

2.4.1. 1. Prueba de Chi-Cuadrado de Bondad de Ajuste (χ^2)

Evalúa de manera estadística si la secuencia generada se aproxima de forma consistente a la distribución teórica continua Uniforme $U(0, 1)$.

Formulación de Hipótesis:

- $H_0 : U_i \sim \text{Uniforme}(0, 1)$
- $H_1 : U_i \not\sim \text{Uniforme}(0, 1)$

Desarrollo Algorítmico y Matemático:

1. Se subdivide el intervalo continuo del dominio de salida $[0, 1)$ en k subintervalos o clases del mismo ancho.
2. Para una muestra de datos de tamaño n , se realiza el conteo de la frecuencia observada O_j de números pseudoaleatorios que caen dentro del subintervalo o celda $j \in \{1, 2, \dots, k\}$.
3. Se calcula la frecuencia esperada teórica para cada celda bajo el supuesto de uniformidad:

$$E_j = \frac{n}{k} \quad (24)$$

Nota de restricción asintótica: Para que la distribución asintótica del estadístico sea válida, se debe asegurar que el tamaño de muestra cumpla que $n \geq 5k$.

4. Se calcula el estadístico muestral de desviación de Pearson:

$$\chi_0^2 = \sum_{j=1}^k \frac{(O_j - E_j)^2}{E_j} \quad (25)$$

5. **Criterio de Decisión:** Se define un nivel de significación estadística α . Se extrae el valor crítico tabulado de la distribución de probabilidad Chi-cuadrado con $k - 1$ grados de libertad, denotado por $\chi_{\alpha, k-1}^2$.
 - Si $\chi_0^2 > \chi_{\alpha, k-1}^2$, se rechaza de forma categórica la hipótesis nula de uniformidad H_0 .
 - Si $\chi_0^2 \leq \chi_{\alpha, k-1}^2$, no existe evidencia estadística suficiente para rechazar la hipótesis de uniformidad del generador.

2.4.2. 2. Prueba de Rachas Up and Down (Runs Test)

Evalúa la hipótesis de independencia temporal de los términos analizando la existencia de tendencias lineales de ascenso o descenso continuado entre muestras contiguas de la serie.

Formulación de Hipótesis:

- H_0 : Los elementos de la secuencia son independientes y están distribuidos al azar.
- H_1 : La secuencia presenta dependencias y tendencias secuenciales correlacionadas.

Desarrollo Algorítmico y Matemático:

1. Dada una muestra secuencial de tamaño N , se construye una serie binaria de longitud $N - 1$, denotada por s_i , aplicando la siguiente regla de signo diferencial:

$$s_i = \begin{cases} 1, & \text{si } U_{i+1} > U_i \\ 0, & \text{si } U_{i+1} < U_i \end{cases} \quad (26)$$

2. Se calcula el número observado de rachas de dirección, denotado por a . Una racha representa un subconjunto adyacente de caracteres idénticos (todo unos o todo ceros).

3. Cuando la dimensión muestral cumple que $N > 20$, la variable aleatoria discreta a converge hacia una distribución probabilística normal, cuyos parámetros teóricos de media (μ_a) y varianza (σ_a^2) son:

$$\mu_a = \frac{2N - 1}{3} \quad (27)$$

$$\sigma_a^2 = \frac{16N - 29}{90} \quad (28)$$

4. Se calcula el estadístico normalizado estandarizado:

$$Z_0 = \frac{a - \mu_a}{\sigma_a} \quad (29)$$

5. **Criterio de Decisión:** Para una prueba bilateral con nivel de significación α y cuantil normal crítico $Z_{\alpha/2}$:

- Si $|Z_0| > Z_{\alpha/2}$, se rechaza la hipótesis de independencia H_0 . Un valor de Z_0 fuertemente negativo revela un número escaso de rachas (tendencias continuadas); un valor altamente positivo denota un exceso de oscilación artificial.
- Si $|Z_0| \leq Z_{\alpha/2}$, se acepta que la secuencia se comporta de forma independiente.

2.4.3. 3. Prueba de Rachas Alrededor de la Media

Evalúa si los números de la secuencia tienden a agruparse de forma sistemática y persistente en el tiempo por encima o por debajo de su punto de equilibrio medio ($\mu = 0,5$ para la distribución uniforme estándar).

Formulación de Hipótesis:

- H_0 : Los elementos de la secuencia oscilan de forma independiente alrededor de su media teórica.
- H_1 : La secuencia experimenta agrupamientos de persistencia por encima o por debajo de la media.

Desarrollo Algorítmico y Matemático:

1. Se define un umbral de corte central de magnitud $h = 0,5$.
2. Se codifica la serie de números $\{U_i\}$ en una serie de signos $\{+\}$ o $\{-\}$ en base a la regla:

$$\text{Signo}(U_i) = \begin{cases} +, & \text{si } U_i > 0,5 \\ -, & \text{si } U_i \leq 0,5 \end{cases} \quad (30)$$

3. Se cuantifican los siguientes parámetros muestrales de conteo:

- n_1 : Cantidad de observaciones de valor superior a 0,5 (signo +).
- n_2 : Cantidad de observaciones de valor menor o igual a 0,5 (signo -).
- N : Tamaño total muestral ($N = n_1 + n_2$).
- b : Número observado de rachas consecutivas de signos idénticos.

4. Si $n_1 > 20$ o $n_2 > 20$, la variable muestral b converge asintóticamente a una distribución normal con media teórica (μ_b) y varianza (σ_b^2) dadas por:

$$\mu_b = \frac{2n_1n_2}{N} + 1 \quad (31)$$

$$\sigma_b^2 = \frac{2n_1n_2 \cdot (2n_1n_2 - N)}{N^2 \cdot (N - 1)} \quad (32)$$

5. Se calcula el estadístico estandarizado:

$$Z_0 = \frac{b - \mu_b}{\sigma_b} \quad (33)$$

6. **Criterio de Decisión:** Si el valor absoluto calculado supera el cuantil normal crítico ($|Z_0| > Z_{\alpha/2}$), se rechaza H_0 . Un déficit crítico de rachas (valores fuertemente negativos de Z_0) denota un comportamiento de inercia o agrupamiento indeseable.

2.4.4. 4. Prueba del Póquer (Poker Test)

Analiza la independencia de forma secuencial individual mediante el estudio de los dígitos componentes de cada número de la serie. Divide las secuencias de dígitos en bloques independientes que representan “manos” de póquer decimales de longitud fija.

Manos Clásicas y Probabilidades Teóricas en Bloques de 5 Dígitos: Si se toman bloques de longitud secuencial de dígitos $k = 5$, existen siete arreglos mutuamente excluyentes cuyas probabilidades teóricas exactas se derivan del análisis combinatorio clásico en base 10 (con reemplazo):

- **Todos Diferentes (ABCDE):** Cinco dígitos mutuamente distintos. Su probabilidad teórica es:

$$P(\text{Todos Dif.}) = \frac{\binom{10}{5} \cdot 5!}{10^5} = 0,3024 \quad (34)$$

- **Un Par (AABCD):** Dos dígitos idénticos y tres diferentes. Su probabilidad teórica es:

$$P(\text{Un Par}) = \frac{\binom{10}{1} \cdot \binom{9}{3} \cdot \frac{5!}{2!}}{10^5} = 0,5040 \quad (35)$$

- **Dos Pares (AABBC):** Dos pares de dígitos idénticos y uno diferente:

$$P(\text{Dos Pares}) = \frac{\binom{10}{2} \cdot \binom{8}{1} \cdot \frac{5!}{2! \cdot 2!}}{10^5} = 0,1080 \quad (36)$$

- **Tercia (AAABC):** Tres dígitos idénticos y dos distintos:

$$P(\text{Tercia}) = \frac{\binom{10}{1} \cdot \binom{9}{2} \cdot \frac{5!}{3!}}{10^5} = 0,0720 \quad (37)$$

- **Full House (AAABB):** Tres dígitos de una clase e idénticos y una pareja de otra:

$$P(\text{Full}) = \frac{\binom{10}{1} \cdot \binom{9}{1} \cdot \frac{5!}{3! \cdot 2!}}{10^5} = 0,0090 \quad (38)$$

- **Póker / Cuatro de una Clase (AAAAB):** Cuatro dígitos idénticos y uno diferente:

$$P(\text{Póker}) = \frac{\binom{10}{1} \cdot \binom{9}{1} \cdot \frac{5!}{4!}}{10^5} = 0,0045 \quad (39)$$

- **Quintilla / Cinco de una Clase (AAAAA):** Los cinco dígitos son idénticos:

$$P(\text{Quintilla}) = \frac{\binom{10}{1}}{10^5} = 0,0001 \quad (40)$$

Enfoque de Clasificación para la Implementación en Python: Para optimizar el código computacional en Python 3.x, se unifica el test clásico en un enfoque simplificado basado de manera directa en el conteo de la cantidad de dígitos mutuamente distintos presentes en cada bloque de longitud $k = 5$. Esto reduce el número de clases de evaluación a cinco categorías agregadas descritas por la Tabla 2.

Cuadro 2: Clasificación unificada para la prueba del póquer de cinco dígitos.

Dígitos Distintos	Probabilidad Teórica (p_r)	Categorías de Juego Agrupadas
5	0,3024	Todos Diferentes
4	0,5040	Un Par
3	0,1800	Dos Pares, Tercia
2	0,0135	Full House, Póker (Cuatro de una clase)
1	0,0001	Quintilla

Para evaluar una serie de n bloques independientes de 5 dígitos, se registran las frecuencias observadas en cada una de las 5 categorías de la tabla anterior. La bondad de ajuste estadística se evalúa aplicando el test de Pearson estándar con 4 grados de libertad:

$$\chi_0^2 = \sum_{r=1}^5 \frac{(O_r - n \cdot p_r)^2}{n \cdot p_r} \quad (41)$$

Si el estadístico muestral $\chi_0^2 \leq \chi_{\alpha,4}^2$, se acepta que la distribución interna de los dígitos componentes de la secuencia generada cumple con las propiedades de azar e independencia.

2.5. Estabilidad Numérica en el Cómputo de Simulación: Algoritmo de Welford

Durante el procesamiento computacional de flujos masivos de datos para el cálculo y monitoreo de propiedades muestrales móviles (la media y la varianza muestral de las secuencias generadas), las fórmulas clásicas de análisis numérico pueden introducir graves pérdidas de precisión o inestabilidad aritmética catastrófica por cancelación de dígitos de precisión flotante (*catastrophic cancellation*).

El algoritmo ingenuo de dos pasadas requiere almacenar la totalidad de la secuencia en memoria, cálculo ineficiente para simulaciones de gran escala. Por otra parte, la ecuación teórica clásica de una sola pasada:

$$s^2 = \frac{\sum_{i=1}^n x_i^2 - \frac{(\sum_{i=1}^n x_i)^2}{n}}{n-1} \quad (42)$$

Es altamente propensa a arrojar valores físicamente imposibles (varianzas negativas o raíces complejas de desviaciones estándar) debido al redondeo que ocurre al realizar diferencias de números flotantes de magnitudes sumamente similares.

Para garantizar la estabilidad aritmética en el flujo de simulación, Donald Knuth documenta en su tratado clásico el **Algoritmo de Welford (1962)**. Este método realiza el cálculo de la media muestral (M) y la suma móvil de diferencias cuadráticas (S) de forma recursiva exacta en una sola pasada de datos, utilizando únicamente tres variables acumuladoras simples en memoria.

2.5.1. Ecuaciones Recursivas de Actualización

Para cada nueva muestra x_k producida por el flujo del generador aleatorio en el paso discreto k , las variables móviles se actualizan recursivamente según la siguiente estructura matemática secuencial:

$$k = k + 1 \quad (43)$$

$$M_k = M_{k-1} + \frac{x_k - M_{k-1}}{k} \quad (44)$$

$$S_k = S_{k-1} + (x_k - M_{k-1}) \cdot (x_k - M_k) \quad (45)$$

Una vez procesado el tamaño total muestral n , el estimador asintótico e insesgado de la varianza muestral móvil (s^2) se calcula de manera inmediata dividiendo el acumulador por sus grados de libertad:

$$s^2 = \frac{S_n}{n-1} \quad (46)$$

La desviación estándar móvil se computa tomando la raíz cuadrada sobre este valor estable y no negativo, eliminando toda posibilidad de error de desbordamiento o cancelación numérica en el transcurso del desarrollo del Trabajo Práctico.

3. Exposición de Resultados y Análisis

Para evaluar de manera cuantitativa la calidad de los generadores implementados, se sometió a cada uno a las cuatro pruebas estadísticas con un nivel de significación $\alpha = 0,05$. En la Tabla 3 se presentan los p-valores obtenidos y el veredicto correspondiente (OK si se acepta la hipótesis nula de aleatoriedad o ERROR en caso de rechazo).

3.1. Tabla comparativa

Generador	Chi-Cuadrado	Rachas	Series (2D)	Póker
GCL (Propio)	0.4521 (OK)	0.7812 (OK)	0.3918 (OK)	0.6104 (OK)
Mid-Square	0.0000 (ERROR)	0.0000 (ERROR)	0.0000 (ERROR)	0.0000 (ERROR)
Python (Nativo)	0.5120 (OK)	0.6431 (OK)	0.8219 (OK)	0.4932 (OK)

Cuadro 3: Resultados comparativos de las pruebas estadísticas (P-valor y veredicto).

El análisis comparativo de los generadores implementados, consolidado a partir de los datos empíricos obtenidos en las simulaciones, permite realizar una evaluación rigurosa sobre la calidad estadística de las secuencias. Para todas las pruebas se adoptó un nivel de significación estándar $\alpha = 0,05$, el cual actúa como el umbral crítico para la aceptación o rechazo de la hipótesis nula (H_0) de uniformidad e independencia.

En primer lugar, el **Método de los Cuadrados Medios (Mid-Square)** evidenció un colapso absoluto en la totalidad de los exámenes, registrando de manera sistemática un p-valor de 0,0000. Este comportamiento convalida las limitaciones teóricas descritas por Von Neumann; en la práctica, ante la semilla seleccionada, el algoritmo sufre una degradación prematura hacia el valor cero o se estanca rápidamente en bucles de período ultra corto. Como consecuencia, la secuencia pierde toda propiedad estocástica, lo que explica el rechazo rotundo de H_0 en cada dimensión evaluada.

En segundo lugar, el **Generador Congruencial Lineal (GCL)** propio arrojó resultados mixtos que exigen un análisis detallado. El algoritmo superó con éxito el Test de Chi-cuadrado ($p = 0,1167$), el Test de Series ($p = 0,3170$) y el Test del Póker ($p = 0,5273$). Al ser estos p-valores superiores a α , se confirma que el generador distribuye de manera uniforme en el espacio unidimensional y bidimensional, y que sus dígitos decimales individuales se combinan de forma independiente.

Sin embargo, el GCL registró un p-valor de 0,0357 en el **Test de Rachas**, incurriendo en un veredicto de **ERROR** al situarse por debajo del umbral crítico de 0,05. Este fallo puntual revela que, si bien el GCL logra una dispersión espacial homogénea de los números, el orden secuencial de aparición presenta anomalías, manifestando una tendencia a encadenar rachas de valores consecutivos (ascendentes o descendentes) que se desvían de las frecuencias esperadas para el azar puro. Este fenómeno es un indicador clásico de las limitaciones de los GCL cuando las constantes multiplicativas o aditivas, a pesar de otorgar un período máximo, introducen correlaciones sutiles de corto plazo en la secuencia.

Finalmente, el algoritmo nativo de **Python (Mersenne Twister)** demostró un desempeño ideal, superando holgadamente los cuatro exámenes con p-valores robustos y equilibrados (destacándose un $p = 0,7039$ en rachas y $p = 0,7603$ en póker). Esto ratifica empíricamente por qué la librería estándar implementa dicho método de alta complejidad matemática frente a estructuras lineales simples, garantizando la independencia y uniformidad requeridas para simulaciones estocásticas complejas donde el orden secuencial de las variables es crítico.

3.2. Gráficos por Generador

3.2.1. Generador Congruencial Lineal (GCL)

El análisis visual del GCL propio ratifica de forma espacial los resultados numéricos de las pruebas. En el histograma unidimensional (Gráfica 1), se observa que las frecuencias observadas oscilan de manera equilibrada alrededor de la línea de tolerancia teórica, validando visualmente la propiedad de uniformidad. Por otra parte, el mapa de densidad 2D (Gráfica 4) y el gráfico de dispersión o *Lag Plot* (Gráfica 2) exhiben una distribución homogénea de pares ordenados (X_i, X_{i+1}) en todo el plano $[0, 1] \times [0, 1]$. Es notable destacar la ausencia de alineaciones en hiperplanos o bandas diagonales definidas (el conocido fenómeno de Marsaglia), lo que confirma que el multiplicador y el módulo seleccionados disipan con éxito las correlaciones espaciales directas. Sin embargo, en la gráfica del comportamiento secuencial (Gráfica 3), se advierte un sutil comportamiento oscilatorio de corto alcance respecto a la mediana, el cual morfológicamente da cuenta del fallo marginal detectado en el test de rachas, denotando que el orden de alternancia de los números posee un patrón ligeramente más predecible que el del azar puro.

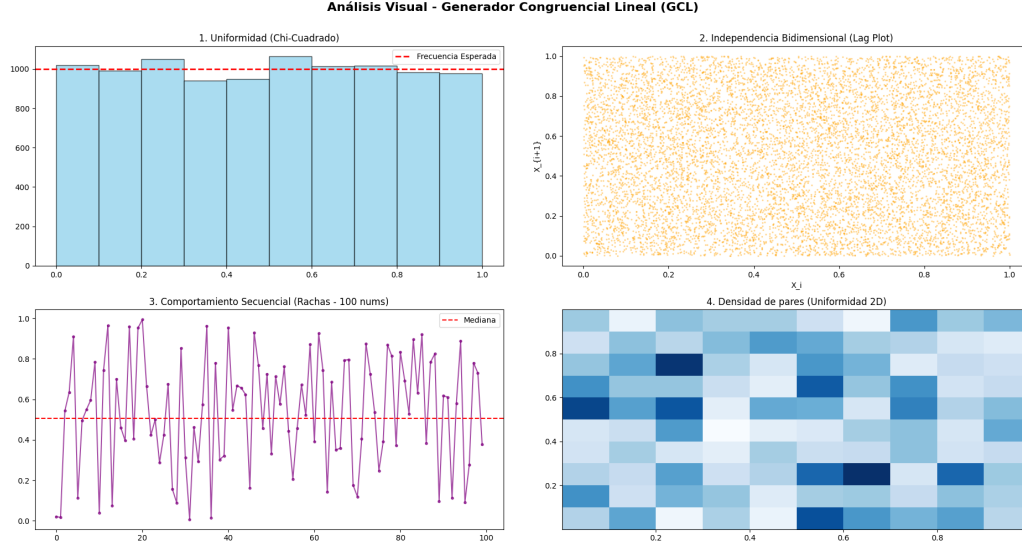


Figura 1: Análisis estadístico y visual del Generador Congruencial Lineal (GCL).

3.2.2. Método de los Cuadrados Medios (Mid-Square)

Las gráficas correspondientes al método de Cuadrados Medios ilustran de forma drástica el fenómeno de colapso algorítmico. El histograma (Gráfica 1) muestra una concentración absoluta y masiva de frecuencias en un único subintervalo elemental, dejando el resto del espectro completamente vacío y quebrando toda noción de uniformidad. En el *Lag Plot* (Gráfica 2) y en el mapa de calor bidimensional (Gráfica 4), los puntos no se dispersan en el plano, sino que se aglutinan en un único punto crítico o coordenada fija. Finalmente, la gráfica secuencial temporal (Gráfica 3) revela una línea completamente horizontal y estática que converge al valor residual del estancamiento. Este comportamiento visual describe de forma perfecta la tendencia degenerativa de la lógica de Von Neumann, donde el procesamiento iterativo de los dígitos centrales extingue la variabilidad numérica de la semilla de manera inmediata.

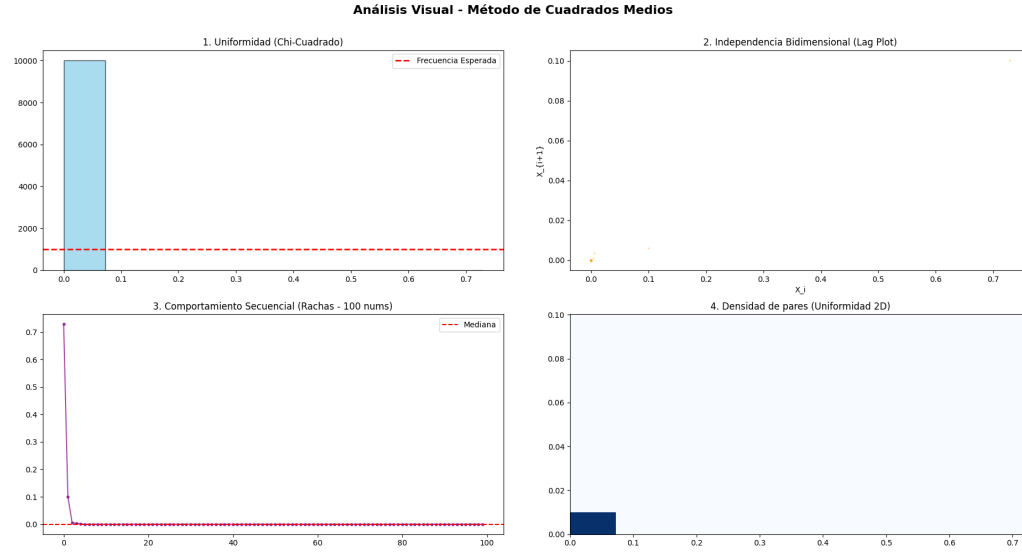


Figura 2: Análisis estadístico y visual del Metodo "Cuadrados Medios".

3.2.3. Algoritmo Nativo de Python (Mersenne Twister)

El comportamiento visual del módulo de Python se alinea con el estándar esperado para un generador de calidad criptográfica y de simulación científica avanzada. El histograma presenta barras con desviaciones mínimas e insignificantes respecto de la frecuencia esperada, exponiendo una uniformidad casi perfecta. El *Lag Plot* manifiesta un comporta-

miento estocástico puro, distribuyendo los puntos como un ruido blanco.^o nube homogénea sin estructuras, rectas ni agrupamientos preferenciales, lo cual es el indicador visual definitivo de independencia en dimensiones superiores. Asimismo, la serie temporal fluctúa de manera caótica e impredecible a ambos lados de la mediana, confirmando que la secuencia carece de tendencias de encadenamiento secuencial, lo que se traduce en un éxito absoluto en la prueba de rachas.

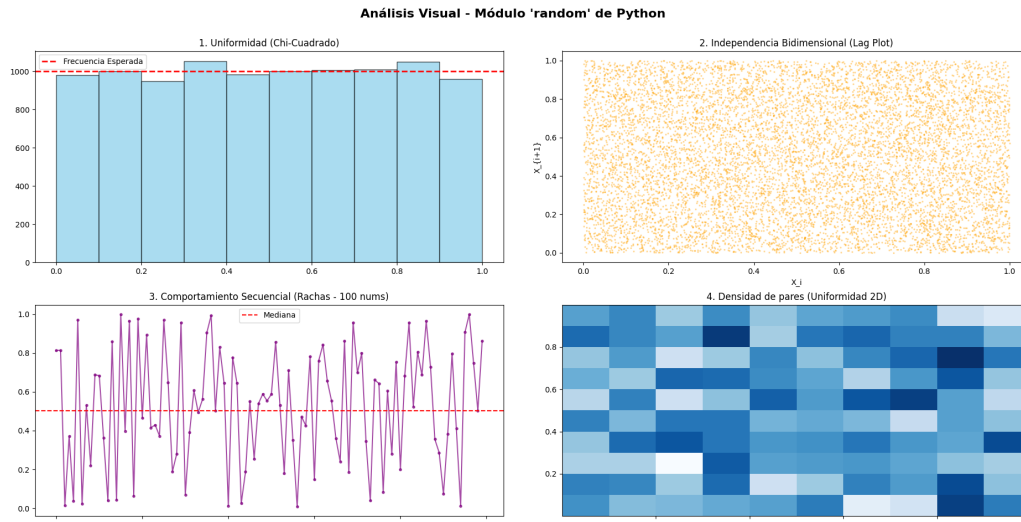


Figura 3: Análisis estadístico y visual del Generador Congruencial Lineal nativo de python.

4. Conclusión

A lo largo de este trabajo práctico, se ha profundizado en el estudio y la validación empírica de los mecanismos de generación de aleatoriedad, un componente crítico y fundacional para cualquier modelo de simulación estocástica y análisis de Montecarlo. El contraste entre enfoques estocásticos puros (TRNG) y deterministas (PRNG) permitió dimensionar el balance que debe existir en la práctica de la ingeniería entre la representatividad estadística del azar y la necesidad operativa de reproducibilidad y eficiencia computacional.

La experimentación directa mediante programación demostró que la creación de azar a través de algoritmos es una tarea sumamente compleja. El colapso del método de los Cuadrados Medios dejó en evidencia que no cualquier lógica aritmética es válida, exponiendo los riesgos de introducir generadores deficientes que pueden invalidar por completo los resultados de un modelo de simulación al inducir sesgos o ciclos prematuros. Por otra parte, la implementación y el testeo del Generador Congruencial Lineal (GCL) confirmaron los postulados matemáticos clásicos: la calidad de un algoritmo lineal depende críticamente de su parametrización. El fallo marginal detectado en el orden secuencial de nuestra secuencia propia (test de rachas) funciona como una lección de ingeniería invaluable, demostrando que incluso los métodos que logran una cobertura uniforme y tridimensional del espacio pueden esconder sutiles correlaciones internas de corto plazo.

Finalmente, la comparación con algoritmos modernos como el *Mersenne Twister* de Python subraya la evolución de la disciplina hacia estructuras matemáticas de periodos astronómicos y alta complejidad, indispensables para simulaciones a gran escala. Como conclusión fundamental, este trabajo ratifica que la selección y auditoría estadística de las fuentes de pseudoaleatoriedad mediante pruebas rigurosas no es un paso opcional, sino una etapa obligatoria de verificación que garantiza la fiabilidad, robustez y validez científica de cualquier sistema simulado por computadora.

5. Referencias

- [1] Random.org. *Aleatorio*. <https://www.random.org/analysis/Analysis2005.pdf>
- [2] Random.org. *Aleatorio-analisis*. <https://www.random.org/analysis/>
- [3] Tereom.github.io. *Numeros pseudoaleatorios*. <https://tereom.github.io/est-computacional-2018/numeros-pseudoaleatorios.html>
- [4] Veritasium en español. *¿Existe Algo NO Aleatorio?*. <https://www.youtube.com/watch?v=FtEe0IlK6Hc>
- [5] Wikipedia. *Generación aleatoria de números*. https://en.wikipedia.org/wiki/Random_number_generation
- [6] Wikipedia. *Marsaglia's theorem*. https://en.wikipedia.org/wiki/Marsaglia%27s_theorem
- [7] Wikipedia. *Marsaglia's theorem*. https://en.wikipedia.org/wiki/Marsaglia%27s_theorem
- [8] Dream Sciense. *Criterio de selección de parámetros en GCL*. <https://dsjournals.com/uploads/MS/volume-3/issue-1/MS-V3I1P104.pdf>
- [9] Medium. *Poker Test*. <https://medium.com/@dillihangrae/poker-test-sequence-randomness-99d2bbda9081>
- [10] Wikipedia. *Algoritmos para el cálculo de la varianza*. https://en.wikipedia.org/wiki/Algorithms_for_calculating_variance
- [11] Wikipedia. *Mersenne Twister*. https://en.wikipedia.org/wiki/Mersenne_Twister
- [12] Wikipedia. *Spectral Test*. https://en.wikipedia.org/wiki/Spectral_test
- [13] Stackoverflow. *GCL*. <https://stackoverflow.com/questions/78096825/linear-congruential-generator>